

Core Language Working Group "ready" Issues for the June, 2025 meeting

References in this document reflect the section and paragraph numbering of document [WG21 N5008](#).

2843. Undated reference to Unicode makes C++ a moving target

Section: Clause 2 [\[intro.refs\]](#) **Status:** ready **Submitter:** Jonathan Wakely **Date:** 2024-01-05

Clause 2 [\[intro.refs\]](#) paragraph 1.10 uses an undated reference for the Unicode standard, meaning that a published (and otherwise frozen) standard for C++ (e.g. C++23) will implicitly refer to a new revision of the Unicode standard the moment such is issued:

... For undated references, the latest edition of the referenced document (including any amendments) applies.

- ...
- The Unicode Consortium. The Unicode Standard. Available from:
<https://www.unicode.org/versions/latest/>

This situation is strictly worse than the lack of support for certain scripts or languages, which can be rectified by updating the reference to Unicode in the next revision of the C++ standard, as is regularly done with any other missing language feature deemed worth addressing.

Possible resolution [SUPERSEDED]:

Change in Clause 2 [\[intro.refs\]](#) paragraph 1.10 as follows:

- ...
- The Unicode Consortium. The Unicode Standard, **Version 15.0**. Available from:
~~<https://www.unicode.org/versions/latest/>~~ <https://www.unicode.org/versions/Unicode15.0.0/>

Additional notes (January, 2024)

Forwarded to SG16 and LWG by decision of the CWG chair, via [paper issue 1736](#).

SG16 2024-01-10

SG16 has consensus to have a dated reference to Unicode in the "Normative references", indicating a minimum version, and add permission to implement an implementation-defined later version.

Possible resolution [SUPERSEDED]:

1. Change in Clause 2 [\[intro.refs\]](#) paragraph 1.10 as follows:

- ...
- The Unicode Consortium. The Unicode Standard, **Version 15.0**. Available from:
~~<https://www.unicode.org/versions/latest/>~~<https://www.unicode.org/versions/Unicode15.0.0/>

2. Add a paragraph before 4.1.1 [[intro.compliance.general](#)] paragraph 8 as follows:

A conforming implementation may implement an implementation-defined version of the Unicode Standard that is a later version than the one referenced in Clause 2 [[intro.refs](#)].

A conforming implementation may have extensions (including additional library functions), provided they do not alter the behavior of any well-formed program.

CWG 2024-01-19

CWG took note of the issue. No objections were raised regarding the suggested direction.

EWG 2024-03-18

Allow implementation-defined Unicode version, but require at least version 15.1.

Possible resolution [SUPERSEDED]:

1. Change in Clause 2 [[intro.refs](#)] paragraph 1.10 as follows:

- ...
- The Unicode Consortium. The Unicode Standard, **Version 15.1**. Available from:
~~<https://www.unicode.org/versions/latest/>~~<https://www.unicode.org/versions/Unicode15.1.0/>

2. Add a paragraph before 4.1.1 [[intro.compliance.general](#)] paragraph 8 as follows:

A conforming implementation may implement an implementation-defined version of the Unicode Standard that is a later version than the one referenced in Clause 2 [[intro.refs](#)].

A conforming implementation may have extensions (including additional library functions), provided they do not alter the behavior of any well-formed program.

Additional notes (March, 2024)

More drafting to rebase Annex E on Unicode 15.1 is needed.

Additional notes (May, 2025)

See also P3727R0.

Proposed resolution (approved by CWG 2025-06-20):

1. Change in Clause 2 [[intro.refs](#)] paragraph 1.10 as follows:

- ...
- The Unicode Consortium. The Unicode Standard, **Version 15.1**. Available from:
~~<https://www.unicode.org/versions/latest/>~~<https://www.unicode.org/versions/Unicode15.1.0/>

2. Add a paragraph before 4.1.1 [[intro.compliance.general](#)] paragraph 8 as follows:

A conforming implementation may use an implementation-defined version of the Unicode Standard that is a later version than the one referenced in Clause 2 [[intro.refs](#)].

A conforming implementation may have extensions (including additional library functions), provided they do not alter the behavior of any well-formed program.

3. Remove E.2.2 [\[uaxid.def.rfmt\]](#):

E.2.2.1 R1a Restricted format characters [\[uaxid.def.rfmt\]](#)

If an implementation of UAX #31 wishes to allow format characters such as u+200d zero-width joiner or u+200e zero-width non-joiner it must define a profile allowing them, or describe precisely which combinations are permitted.

C++ does not allow format characters in identifiers, so this does not apply.

2898. Clarify implicit conversion sequence from *cv T* to *T*

Section: 12.2.4.2.1 [\[over.best.ics.general\]](#) **Status:** ready **Submitter:** Brian Bi **Date:** 2024-05-30

(From submission [#544](#).)

Subclause 12.2.4.2.1 [\[over.best.ics.general\]](#) paragraph 7 specifies:

When the parameter has a class type and the argument expression has the same type, the implicit conversion sequence is an identity conversion. When the parameter has a class type and the argument expression has a derived class type, the implicit conversion sequence is a derived-to-base conversion from the derived class to the base class. A derived-to-base conversion has Conversion rank (12.2.4.2.2 [\[over.ics.scs\]](#)).

Does "the same type" imply that differences in cv-qualification are significant? 12.2.4.2.1 [\[over.best.ics.general\]](#) paragraph 6 appears to differ:

... Any difference in top-level cv-qualification is subsumed by the initialization itself and does not constitute a conversion. [Example 2: A parameter of type *A* can be initialized from an argument of type *const A*. The implicit conversion sequence for that case is the identity sequence; it contains no "conversion" from *const A* to *A*. -- end example]

The example appears to reflect the intent; the normative wording should be clarified.

Proposed resolution (no objections during CWG reflector review starting 2025-03-11):

1. Change in 12.2.4.2.1 [\[over.best.ics.general\]](#) paragraph 6, moving the amended example to paragraph 7:

... Any difference in top-level cv-qualification is subsumed by the initialization itself and does not constitute a conversion. [Example 2: A parameter of type *A* can be initialized from an argument of type *const A*. The implicit conversion sequence for that case is the identity sequence; it contains no "conversion" from *const A* to *A*. -- end example]

2. Change in 12.2.4.2.1 [\[over.best.ics.general\]](#) paragraph 7 as follows:

When the parameter has a class type and the argument expression has the same type the cv-unqualified version of the type of the argument expression is the same as the parameter type, the implicit conversion sequence is an identity conversion. When the parameter has a class type and the argument expression has a (possibly cv-qualified) derived class type, the implicit conversion sequence is a derived-to-base conversion from the derived class to the base class. A derived-to-base conversion has Conversion rank (12.2.4.2.2 [\[over.ics.scs\]](#)). [Example: An implicit conversion sequence from an argument of type *const A* to a parameter of type *A* can be formed, even if overload resolution for copy-initialization of *A* from the argument would not find a viable function (12.2.2.4 [\[over.match.ctor\]](#), 12.2.3 [\[over.match.viable\]](#)). The implicit

conversion sequence for that case is the identity sequence; it contains no "conversion" from const A to A. -- end example]

2982. Deduction in *type-constraints*

Section: 13.10.3.7 [[temp.deduct.decl](#)] **Status:** ready **Submitter:** Richard Smith **Date:** 2023-03-30

Consider:

```
template<typename T, typename U> concept C = true;
template<typename T> C<T> auto f() { return 0; }
template C<int> auto f();
```

Is that valid? Or is the following needed:

```
template C<int> auto f<int>();
```

The specification neither discusses deduction from a *type-constraint* nor treating it as a non-deduced context.

Proposed resolution (no objections during CWG reflector review starting 2025-03-18) (approved by CWG 2025-06-20):

Change in 13.10.3.6 [[temp.deduct.type](#)] paragraph 5 as follows:

The non-deduced contexts are:

- The *nested-name-specifier* of a type that was specified using a *qualified-id*.
- A *pack-index-specifier* or a *pack-index-expression*.
- A *type-constraint*.
- The *expression* of a *decltype-specifier*.
- ...

2985. Unclear rules for reference initialization with conversion

Section: 9.5.4 [[decl.init.ref](#)] **Status:** ready **Submitter:** Anoop Rana **Date:** 2024-08-13

(From submission [#596](#).)

It is unclear whether 9.5.4 [[decl.init.ref](#)] bullet 5.4.1 intends list-initialization or not-list-initialization when it talks about direct-initialization.

Furthermore, the following example was well-formed before the resolution of issue 1604 was applied:

```
struct X { };

struct Y : X {};

struct Z {
    operator const Y () const;
};
```

```
Z z;  
X&& r = z; // #1, ill-formed; was well-formed before CWG1604
```

Possible resolution (January, 2025) [SUPERSEDED]:

Change in 9.5.4 [dcl.init.ref] bullet 5.4.1 as follows:

If T1 or T2 is a class type, user-defined conversions are considered using the rules for copy-initialization of an object of type “cv1 T1” by user-defined conversion (9.5 [dcl.init], 12.2.2.5 [over.match.copy], 12.2.2.6 [over.match.conv]); the program is ill-formed if the corresponding non-reference copy-initialization would be ill-formed. The result **E** of the call to the conversion function, as described for the non-reference copy-initialization, is then used to direct-initialize the reference **using the form (E); if E is a prvalue, its cv-qualification is adjusted to cv1**. For this direct-initialization, user-defined conversions are not considered.

Additional notes (February, 2025)

Permitting a binding of X&& to a const Y seems ill-advised; the change effected by issue 1604 in that regard is intended.

In more detail, copy-initializing a T object from a cv T prvalue succeeds:

```
struct X { X() = default; X(X&&) = delete; };  
using CX = const X;  
X x = CX(); // OK, default-initializes x
```

However, even with guaranteed copy elision, the pre-CWG1604 model does not handle derived classes appropriately when reference binding:

```
struct X {  
    X() = default;  
    X(X&&) = delete;  
};  
struct Y : X {};  
struct Z {  
    operator Y() { return Y(); }  
};  
X&& x = Z();
```

In this case, the rvalue reference x should bind directly to the Y materialized prvalue; there should never be an attempt to copy-initialize an X from Z to satisfy the reference binding. However, such direct reference binding would not be expected to work for a const Y prvalue.

Proposed resolution (February, 2025) (approved by CWG 2025-06-20):

Change in 9.5.4 [dcl.init.ref] bullet 5.4.1 as follows:

If T1 or T2 is a class type, user-defined conversions are considered using the rules for copy-initialization of an object of type “cv1 T1” by user-defined conversion (9.5 [dcl.init], 12.2.2.5 [over.match.copy], 12.2.2.6 [over.match.conv]); the program is ill-formed if the corresponding non-reference copy-initialization would be ill-formed. The result **E** of the call to the conversion function, as described for the non-reference copy-initialization, is then used to direct-initialize the reference **using the form (E)**. For this direct-initialization, user-defined conversions are not considered.

There is some ancient non-operational wording in 7.6.1.9 [[expr.static.cast](#)]. It should be removed.

Proposed resolution (no objections during CWG reflector review starting 2025-03-04) (approved by CWG 2025-06-20):

1. Remove in 7.6.1.9 [[expr.static.cast](#)] paragraph 1 as follows:

The result of the expression `static_cast<T>(v)` is the result of converting the expression `v` to type `T`. If `T` is an lvalue reference type or an rvalue reference to function type, the result is an lvalue; if `T` is an rvalue reference to object type, the result is an xvalue; otherwise, the result is a prvalue.
~~The `static_cast` operator shall not cast away constness (7.6.1.11 [[expr.const.cast](#)]).~~

2. Remove 7.6.1.9 [[expr.static.cast](#)] paragraph 6:

~~Otherwise, the inverse of a standard conversion sequence (7.3 [[conv](#)]) not containing an lvalue-to-rvalue (7.3.2 [[conv.lval](#)]), array-to-pointer (7.3.3 [[conv.array](#)]), function-to-pointer (7.3.4 [[conv.func](#)]), null pointer (7.3.12 [[conv.ptr](#)]), null member pointer (7.3.13 [[conv.mem](#)]), boolean (7.3.15 [[conv.bool](#)]), or function pointer (7.3.14 [[conv.fctptr](#)]) conversion, can be performed explicitly using `static_cast`. A program is ill-formed if it uses `static_cast` to perform the inverse of an ill-formed standard conversion sequence. [Example 2:~~

```
struct B { };  
struct D : private B { };  
void f() {  
    static_cast<D*>((B*)0);  
    static_cast<int B::*>((int D::* )0);  
}  
  
—end example]
```

3. Change in 7.6.1.9 [[expr.static.cast](#)] paragraph 7:

~~The~~ **Otherwise, the** lvalue-to-rvalue (7.3.2 [[conv.lval](#)]), array-to-pointer (7.3.3 [[conv.array](#)]), and function-to-pointer (7.3.4 [[conv.func](#)]) conversions are applied to the operand, **and the conversions that can be performed using `static_cast` are listed below. No other conversion can be performed using `static_cast`.** ~~Such a `static_cast` is subject to the restriction that the explicit conversion does not cast away constness (7.6.1.11 [[expr.const.cast](#)]), and the following additional rules for specific cases:~~

4. Remove 7.6.1.9 [[expr.static.cast](#)] paragraph 14:

~~No other conversion can be performed using `static_cast`.~~

3013. Disallowing macros for `#embed` parameters

Subclause 16.4.5.3.3 [[macro.names](#)] paragraph 2 specifies a prohibition against keywords, alternative tokens, and attributes being used as macro names. There is no such protection for parameter names of `#embed`, but the use of *embed-parameters* should be protected there and in the `__has_embed` argument.

Arguably, 16.4.5.3.3 [[macro.names](#)] constrains only programs that use the standard library, because header files of the standard library must be able to rely on a sane environment; see also 16.4.5.3.1 [[reserved.names.general](#)]:

The C++ standard library reserves the following kinds of names: ...

Arguably, standard library headers can be crafted or auto-generated such that they do not make use of `#embed`, or at least not use `#embed` with parameters. In any case, this would be a concern of the standard library, not of the core language.

Proposed resolution (approved by CWG 2025-06-20):

1. Change in 15.1 [\[cpp.pre\]](#) paragraph 3 as follows:

If one of the *pp-tokens* of a `#embed` directive (before macro replacement) is the *identifier* `limit`, `prefix`, `suffix`, or `if_empty` and that *identifier* is defined as a macro (15.7.1 [\[cpp.replace.general\]](#)), the program is ill-formed. Any *embed-prefixed-parameter* is conditionally-supported, with implementation-defined semantics.

2. Change in 15.2 [\[cpp.cond\]](#) paragraph 11 as follows:

Prior to evaluation, macro invocations in the list of preprocessing tokens that will become the controlling constant expression are replaced (except for those macro names modified by the defined unary operator), just as in normal text. **If replacement of macros in the preprocessing tokens following the sequence `__has_embed` (and before a matching) (possibly produced by macro expansion) encounters a preprocessing token that is one of the *identifiers* `limit`, `prefix`, `suffix`, or `if_empty` and that *identifier* is defined as a macro (15.7.1 [\[cpp.replace.general\]](#)), the program is ill-formed.** If the preprocessing token defined is generated as a result of this replacement process or use of the defined unary operator does not match one of the two specified forms prior to macro replacement, the behavior is undefined.

3014. Comma-delimited vs. comma-separated output for `#embed`

Section: 15.4.1 [\[cpp.embed.gen\]](#) **Status:** ready **Submitter:** Hubert Tong **Date:** 2025-02-15

Subclause 15.4.1 [\[cpp.embed.gen\]](#) paragraph 7 specifies "comma-delimited" output for `#embed`. However, that allows for a trailing comma that is undesirable.

Proposed resolution (approved by CWG 2025-06-20):

Change in 15.4.1 [\[cpp.embed.gen\]](#) paragraph 7 and paragraph 8 as follows:

The `#embed` directive is replaced by a ~~comma-delimited~~ **comma-separated** list of integer literals of type `int`, unless otherwise modified by embed parameters (15.4.2 [\[cpp.embed.param\]](#)).

The integer literals in the ~~comma-delimited~~ **comma-separated** list correspond to resource-count consecutive calls to `std::fgetc` (31.13.1 [\[cstdio.syn\]](#)) from the resource, as a binary file. If any call to `std::fgetc` returns EOF, the program is ill-formed.

3015. Handling of *header-names* for `#include` and `#embed`

Section: 15.3 [\[cpp.include\]](#) **Status:** ready **Submitter:** Jens Maurer **Date:** 2025-03-23

There is non-parallel treatment for the *header-name* in `include` vs. `#embed` directives.

First, subclause 5.5 [lex.pptoken] paragraph 4.3 is missing a special-case treatment for #embed.

Second, 15.3 [cpp.include] (and thus 15.4.1 [cpp.embed.gen]) should acknowledge that lexing has completed at that point, and thus talk about *header-name* preprocessing tokens, not about sequences of characters.

Third, 15.4.1 [cpp.embed.gen] paragraph 11 talks about "resource name preprocessing tokens", which do not exist (see 5.5 [lex.pptoken]). Also, it should be clarified this rule applies to the general *pp-tokens* form of #embed only.

Fourth, __has_embed has this aberration:

```
#define stdio nosuch
#if __has_embed(<stdio.h>)    // looks for nosuch.h
#embed <stdio.h>             // looks for stdio.h
#endif
```

For __has_include, this is avoided by using two grammar productions, where the preferred one uses *header-name* (15.2 [cpp.cond]).

Fifth, for __has_include, it is unclear whether only the first (non-macro-expanded) preprocessing token should be eligible for special header-name treatment. There is implementation divergence.

Sixth, for the following example:

```
#embed "foo\" vendor_specific_arg("something else") ...
```

the rule in 5.5 [lex.pptoken] bullet 4.3 would form a *string-literal* (because it consists of a longer sequence of characters), not the *header-name* "foo\".

Seventh, it is unclear how a *q-char-sequence* is supposed to be turned into a *h-char-sequence* when falling back to header search in 15.3 [cpp.include] paragraph 3, given that a *q-char-sequence* might contain a > character, making it not match the production *h-char-sequence*.

Proposed resolution (approved by CWG 2025-06-20):

1. Change in 5.5 [lex.pptoken] bullet 4.3 and add bullets as follows:

- ...
- Otherwise, the next preprocessing token is the longest sequence of characters that could constitute a preprocessing token, even if that would cause further lexical analysis to fail, except that
 - a *header-name* (5.6 [lex.header]) is only formed
 - immediately after the include, embed, or import preprocessing token in a #include (15.3 [cpp.include]), #embed (15.4 [cpp.embed]), or import (15.6 [cpp.import]) directive, respectively, or
 - within a *has-include-expression* immediately after a preprocessing token sequence of __has_include or __has_embed immediately followed by (in a #if, #elif, or #embed directive (15.2 [cpp.cond], 15.4 [cpp.embed]) and
 - a *string-literal* token is never formed when a *header-name* token can be formed.

2. Change in 15.2 [cpp.cond] before paragraph 1:

has-embed-expression:

```
__has_embed ( header-name pp-balanced-token-seqopt )
__has_embed ( header-name-tokens pp-balanced-token-seqopt )
```

3. Change in 15.2 [cpp.cond] paragraph 3 and paragraph 4 as follows:

The second form of *has-include-expression* is considered only if the first form does not match, in which case the preprocessing tokens are processed just as in normal text.

The header or source file identified by the parenthesized preprocessing token sequence in each contained *has-include-expression* is searched for as if that preprocessing token sequence were the *pp-tokens* in of a #include directive, except that no further macro expansion is performed. If such a directive would not satisfy the syntactic requirements of a #include directive, the program is ill-formed. The *has-include-expression* evaluates to 1 if the search for the source file succeeds, and to 0 if the search fails.

4. Change in 15.2 [cpp.cond] paragraph 5 as follows:

The parenthesized *pp-balanced-token-seq* in preprocessing token sequence of each contained *has-embed-expression* is processed as if that *pp-balanced-token-seq* preprocessing token sequence were the *pp-tokens* in the third form of a #embed directive (15.4 [cpp.embed]), except that no further macro expansion is performed. If such a directive would not satisfy the syntactic requirements of a #embed directive, the program is ill-formed. ...

5. Change in 15.3 [cpp.include] paragraph 1 through paragraph 4 as follows:

~~A #include directive shall identify a header or source file that can be processed by the implementation.~~

A *header search* for a sequence of characters searches a sequence of places for a header identified uniquely by that sequence of characters. How the places are determined or the header identified is implementation-defined.

A *source file search* for a sequence of characters attempts to identify a source file that is named by the sequence of characters. The named source file is searched for in an implementation-defined manner. If the implementation does not support a source file search for that sequence of characters, or if the search fails, the result of the source file search is the result of a header search for the same sequence of characters.

A preprocessing directive of the form

```
# include < h-char-sequence > header-name new-line
```

causes the replacement of that directive by the entire contents of the header or source file identified by *header-name*.

If the *header-name* is of the form

```
< h-char-sequence >
```

searches a sequence of implementation-defined places for a header identified uniquely by the specified sequence between the < and > delimiters, and causes the replacement of that directive by the entire contents of the header. How the places are specified or the header identified is implementation-defined a header is identified by a header search for the sequence of characters of the *h-char-sequence*.

A preprocessing directive If the *header-name* is of the form

```
# include " q-char-sequence " new-line
```

causes the replacement of that directive by the entire contents of the source file identified by the specified sequence between the " delimiters. The named source file is searched for in an implementation-defined manner. If this search is not supported, or if the search fails, the directive is reprocessed as if it read

```
# include < h-char-sequence > new-line
```

with the identical contained sequence (including > characters, if any) from the original directive the source file or header is identified by a source file search for the sequence of characters of the *q-char-sequence*.

If a header search fails, or if a source file search or header search identifies a header or source file that cannot be processed by the implementation, the program is ill-formed. [Note: If the header or source file cannot be processed, the program is ill-formed even when evaluating `__has_include`. - end note]

A preprocessing directive of the form

```
# include pp-tokens new-line
```

(that does not match one of the two previous forms) is permitted. The preprocessing tokens after include in the directive are processed just as in normal text (i.e., each identifier currently defined as a macro name is replaced by its replacement list of preprocessing tokens). Then, an attempt is made to form a *header-name* preprocessing token (5.6 [lex.header]) from the whitespace and the characters of the spellings of the resulting sequence of preprocessing tokens; the treatment of whitespace is implementation-defined. If the attempt succeeds, the directive with the so-formed *header-name* is processed as specified for the previous form. Otherwise directive resulting after all replacements does not match one of the two previous forms, the behavior is undefined.

[Note 1: Adjacent *string-literals* are not concatenated into a single *string-literal* (see the translation phases in 5.2 [lex.phases]); thus, an expansion that results in two *string-literals* is an invalid directive. — end note]

The method by which a sequence of preprocessing tokens between a < and a > preprocessing token pair or a pair of " characters is combined into a single header name preprocessing token is implementation-defined.

6. Change in 15.4.1 [cpp.embed.gen] paragraph 1 and paragraph 2 as follows:

A *bracket resource search* for a sequence of characters searches a sequence of places for a resource identified uniquely by that sequence of characters. How the places are determined or the resource identified is implementation-defined.

A *quote resource search* for a sequence of characters attempts to identify a resource that is named by the sequence of characters. The named resource is searched for in an implementation-defined manner. If the implementation does not support a quote resource search for that sequence of characters, or if the search fails, the result of the quote resource search is the result of a bracket resource search for the same sequence of characters.

A preprocessing directive of the form

```
# embed < h-char-sequence > header-name pp-tokensopt new-line
```

causes the replacement of that directive by preprocessing tokens derived from data in the resource identified by *header-name*, as specified below.

If the *header-name* is of the form

```
< h-char-sequence >
```

searches a sequence of implementation-defined places for a resource identified uniquely by the specified sequence between the < and > delimiters. How the places are specified or the resource identified is implementation-defined the resource is identified by a bracket resource search for the sequence of characters of the *h-char-sequence*.

A preprocessing directive **If the header-name is** of the form

```
# embed " q-char-sequence " pp-tokensopt new-line
```

searches for a resource identified by the specified sequence between the " delimiters. The named resource is searched for in an implementation-defined manner. If this search is not supported, or if the search fails, the directive is reprocessed as if it read

```
# embed < h-char-sequence > pp-tokensopt new-line
```

with the identical contained sequence (including > characters, if any) from the original directive. **the resource is identified by a quote resource search for the sequence of characters of the q-char-sequence.**

If a bracket resource search fails, or if a quote or bracket resource search identifies a resource that cannot be processed by the implementation, the program is ill-formed. [Note: If the resource cannot be processed, the program is ill-formed even when processing #embed with `limit(0)` (15.4.2.1 [[cpp.embed.param.limit](#)]) or evaluating `__has_embed`. -- end note]

7. Change in 15.4.1 [[cpp.embed.gen](#)] paragraph 10 and paragraph 11:

(10) A preprocessing directive of the form

```
# embed pp-tokens new-line
```

(that does not match **one of the two previous forms form**) is permitted. The preprocessing tokens after embed in the directive are processed just as in normal text (i.e., each identifier currently defined as a macro name is replaced by its replacement list of preprocessing tokens). **The directive resulting after all replacements of the third form shall match one of the two previous forms. Then, an attempt is made to form a header-name preprocessing token (5.6 [[lex.header](#)]) from the whitespace and the characters of the spellings of the resulting sequence of preprocessing tokens immediately after embed; the treatment of whitespace is implementation-defined. If the attempt succeeds, the directive with the so-formed header-name is processed as specified for the previous form. Otherwise, the program is ill-formed.**

[Note 1: Adjacent *string-literals* are not concatenated into a single *string-literal* (see the translation phases in (5.2 [[lex.phases](#)]))]; thus, an expansion that results in two *string-literals* is an invalid directive. — end note]

Any further processing as in normal text described for the **two previous forms form** is not performed. [Note 2: That is, processing as in normal text happens once and only once for the entire directive. — end note]

(11) [Example 4: If the directive matches the **third second** form, the whole directive is replaced. If the directive matches the first **two forms form**, everything after the name is replaced.

```
#define prefix(ARG) suffix(ARG)
#define THE_ADDITION "teehee"
#define THE_RESOURCE ":3c"
#embed ":3c" prefix(THE_ADDITION)
#embed THE_RESOURCE prefix(THE_ADDITION)
```

```
#define EMPTY
#define X myfile
#define Y rsc
#define Z 42
#embed <myfile.rsc> prefix(Z)
#embed EMPTY <X.Y> prefix(Z)
```

is equivalent to:

```
#embed ":3c" suffix("teehee")  
#embed ":3c" suffix("teehee")
```

```
#embed <myfile.rsc> prefix(42)  
#embed <myfile.rsc> prefix(42)
```

—end example]

~~The method by which a sequence of preprocessing tokens between a < and a > preprocessing token pair or a pair of " characters is combined into a single resource name preprocessing token is implementation-defined.~~

8. Change in 15.7.3 [cpp.stringize] paragraph 2 as follows:

... Otherwise, the original spelling of each preprocessing token in the stringizing argument is retained in the character string literal, except for special handling for producing the spelling of **header-names**, **string-literals**, and **character-literals**: a \ character is inserted before each " and \ character of a **header-name**, **character-literal**, or **string-literal** (including the delimiting " characters). If the replacement that results is not a valid character string literal, the behavior is undefined. ...

3016. Satisfying the syntactic requirements of #include and #embed

Section: 15.2 [cpp.cond] **Status:** ready **Submitter:** Hubert Tong **Date:** 2025-03-24

(From submission [#691](#).)

Subclause 15.2 [cpp.cond] paragraph 4 talks about the "syntactic requirements of a #include directive". However, those requirements are always satisfied, because the only requirement is to have a sequence of preprocessing tokens according to the grammar in 15.1 [cpp.pre].

However, a syntax check for properly forming a *header-name* needs to be retained, because 15.3 [cpp.include] paragraph 4 otherwise would yield undefined behavior.

Proposed resolution (approved by CWG 2025-06-20):

1. Change in 15.2 [cpp.cond] paragraph 4 as follows:

The header or source file identified by the parenthesized preprocessing token sequence in each contained *has-include-expression* is searched for as if that preprocessing token sequence were the *pp-tokens* in a #include directive, except that no further macro expansion is performed. If ~~such a directive would not satisfy the syntactic requirements of a #include directive~~ **the preprocessing token sequence does not consist solely of a header-name or cannot be combined (15.3 [cpp.include]) into a single header-name preprocessing token**, the program is ill-formed. The *has-include-expression* evaluates to 1 if the search for the source file succeeds, and to 0 if the search fails.

2. Change in 15.2 [cpp.cond] paragraph 5 as follows:

The parenthesized *pp-balanced-token-seq* in each contained *has-embed-expression* is processed as if that *pp-balanced-token-seq* were the *pp-tokens* in the third form of a #embed directive (15.4 [cpp.embed]). ~~If such a directive would not satisfy the syntactic requirements of a #embed directive, the program is ill-formed.~~ The *has-embed-expression* evaluates to: ...

3018. Validity of defined in __has_embed

Section: 15.2 [[cpp.cond](#)] **Status:** ready **Submitter:** Hubert Tong **Date:** 2025-03-24

(From submission [#694](#).)

Is the use of defined valid in __has_embed? It is not valid in cases where it would be interpreted by the #if rules in #embed; see 15.4.2.1 [[cpp.embed.param.limit](#)] paragraph 1.

To avoid confusion, defined should also be prohibited inside __has_embed.

Proposed resolution (approved by CWG 2025-06-20):

1. Change in 15.2 [[cpp.cond](#)] paragraph 1 as follows:

The expression that controls conditional inclusion shall be an integral constant expression except that identifiers (including those lexically identical to keywords) are interpreted as described below¹²³ and it may contain zero or more *defined-macro-expressions*, *has-include-expressions*, *has-attribute-expressions*, and/or *has-embed-expressions* as unary operator expressions. **A *defined-macro-expression* shall not appear within a *has-include-expression* or *has-embed-expression*.**

2. Change in 15.4.2.1 [[cpp.embed.param.limit](#)] paragraph 1 as follows:

An *embed-parameter* of the form `limit ( pp-balanced-token-seq )` specifies the maximum possible number of elements in the comma-delimited list. It shall appear at most once in the *embed-parameter-seq*. The **preprocessing** token defined shall not appear in the ~~*constant-expression*~~ ***pp-balanced-token-seq***.

The *pp-balanced-token-seq* is evaluated as a *constant-expression* using the rules as described in conditional inclusion (15.2 [[cpp.cond](#)]), but without being processed as in normal text an additional time.

3020. Missing specification for __has_cpp_attribute(indeterminate)

Section: 15.2 [[cpp.cond](#)] **Status:** ready **Submitter:** Tim Song **Date:** 2025-03-26

(From submission [#695](#).)

P2795R5 (approved in March, 2024) should, but does not, update the definition of __has_cpp_attribute for the new attribute it added.

Proposed resolution (no objections during CWG reflector review starting 2025-03-31) (approved by CWG 2025-06-20):

Add a row to 15.2 [[cpp.cond](#)], table [tab:cpp.cond.ha], as follows:

Attribute	Value
indeterminate	202403L

